

Building — Participant Handout

Modules 4–7 | Day 1 Afternoon + Day 2 Morning

AI-Powered Development with Claude — Corporate Training Course

Keep this document open alongside your laptop during every lab in this block.

MODULE 4 — Reference Card

TaskCo Spec — Quick Reference

Keep this section open during every lab. This is the spec you're building against.

Data model

```
User
  id          String    @id @default(cuid())
  email       String    @unique
  passwordHash String
  name        String
  createdAt   DateTime @default(now())
  projects    Project[]
```

```
Project
  id          String    @id @default(cuid())
  name        String
  description  String?
  color       String    @default("#3b82f6")
  ownerId     String
  owner       User      @relation(...)
  tasks       Task[]
  createdAt   DateTime @default(now())
```

```
Task
  id          String    @id @default(cuid())
  title       String
  description  String?
  status      TaskStatus @default(TODO)
  priority    Priority   @default(MEDIUM)
  dueDate     DateTime?
  projectId   String
  project     Project   @relation(...)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
```

```
enum TaskStatus { TODO, IN_PROGRESS, DONE }
enum Priority    { LOW, MEDIUM, HIGH }
```

API surface

Method	Endpoint	What it does
POST	<code>/auth/register</code>	Create user, return JWT
POST	<code>/auth/login</code>	Verify credentials, return JWT
GET	<code>/auth/me</code>	Return current user (auth required)
GET	<code>/projects</code>	List current user's projects
POST	<code>/projects</code>	Create project
GET	<code>/projects/:id</code>	Get one project (with task count)
PATCH	<code>/projects/:id</code>	Update project
DELETE	<code>/projects/:id</code>	Delete project (cascades to tasks)
GET	<code>/projects/:id/tasks</code>	List tasks (filter by status, priority)
POST	<code>/projects/:id/tasks</code>	Create task
PATCH	<code>/tasks/:id</code>	Update task
DELETE	<code>/tasks/:id</code>	Delete task

Every endpoint: validates with Zod, returns `{ data }` or `{ error }`, scopes by ownership.

Frontend pages

- Login / Register — two forms
- Dashboard — project list with task counts
- Project view — tasks with filter buttons, priority badges, due dates, inline status toggle
- 404 / error states

Out of scope

Email verification, password reset, OAuth, real-time/WebSocket, file uploads, attachments, team collaboration (seam only), notifications, reminders, calendar integration, mobile, deployment, CI/CD.

Docker PostgreSQL command

```
docker run --name taskco-db -e POSTGRES_USER=taskco -e
POSTGRES_PASSWORD=taskco -e POSTGRES_DB=taskco -p 5432:5432 -d postgres:16
```

Your `.env` file should contain:

```
DATABASE_URL="postgresql://taskco:taskco@localhost:5432/taskco"
```

Verify it's running: `docker ps` should show the container. If it's stopped: `docker start taskco-db`.

Lab 1 — TaskCo Kickoff

Three steps: plan, scaffold, connect. By the end, you have a running Fastify server connected to PostgreSQL with a CLAUDE.md and a first Git commit.

Step 1 — Plan in Claude.ai (~15 min)

Goal: Have a planning conversation that produces a concrete folder structure, conventions list, and build order — not vague agreement.

Open Claude.ai. Create a new Project called “TaskCo.” Add the spec from the reference section above (data model, API surface, stack) as project context.

Prompt Checklist — your planning conversation should cover:

- The full stack (TypeScript, Node.js, Fastify, Prisma, PostgreSQL, Zod, bcrypt, jose, Vitest, React, Vite, Tailwind)
- Folder structure — where do routes, services, schemas, types, and tests go? Push for specific paths, not “organize by feature”
- The response envelope — settle on `{ data: ... }` for success and `{ error: ... }` for failure, with consistent shapes
- Naming conventions — file naming (kebab-case? feature-based?), variables (camelCase), types (Pascal-Case)
- Build order — confirm bottom-up: Prisma schema → auth endpoints → projects → tasks → frontend
- Testing strategy — Vitest, where test files live, what kinds of tests
- The ownership model — every project belongs to a user, every query scopes by ownerId
- What’s out of scope — make sure Claude doesn’t plan for features you’re not building

Self-Review Checklist — before moving to Step 2, check:

- Did the conversation produce a concrete folder tree (actual paths like `src/routes/`, `src/services/`), not just principles?
- Did Claude settle on specific conventions, or give you three options? If the latter, choose one now
- Did the build order match bottom-up, or did Claude try to start with the frontend?
- Do you have enough concrete output to write a CLAUDE.md from this conversation?
- Did Claude plan any features that are on the out-of-scope list?

Checkpoint: You have a document with a folder tree, conventions list, response shapes, and build order. Save it — you’ll paste parts of it in Steps 2 and 3.

Step 2 — Scaffold with Claude Code (~30 min)

Goal: A running Fastify hello-world server with Prisma initialized, matching the folder structure from your plan.

Open VS Code. Create a new folder called `taskco` and open it. Open the Claude Code panel.

Prompt Checklist — your scaffolding prompt should make Claude aware of:

- The stack: TypeScript, Node.js, Fastify, pnpm (not npm), Prisma, Vitest
- The folder structure from Step 1 (paste the folder tree from your planning conversation)
- That the entry point is `src/index.ts` — a Fastify server that starts and responds to `GET /` with a health check
- That Prisma should be initialized with a PostgreSQL datasource, but the schema should have no models yet (User model comes in Lab 2)
- That you want `tsconfig.json` configured for strict TypeScript
- That `pnpm dev` should start the server

- What NOT to include: no React, no frontend, no Docker configuration, no CI/CD. Backend scaffolding only

Self-Review Checklist — before accepting, check:

- Does `pnpm install` complete without errors?
- Does `pnpm dev` start a Fastify server? Can you hit `http://localhost:3000` in a browser and get a response?
- Does the folder structure match your plan from Step 1, or did Claude invent its own layout?
- Is `prisma/schema.prisma` present with `provider = "postgresql"` in the datasource?
- Is TypeScript configured in strict mode?
- Did Claude add anything you didn't ask for? (extra folders, frameworks, business logic beyond the health check)

Checkpoint: `pnpm dev` starts Fastify. A browser request to the health check endpoint returns a response. The folder structure matches your plan.

Step 3 — Database, CLAUDE.md, Git (~30 min)

Goal: Connect Prisma to PostgreSQL, write your first CLAUDE.md, and make your first commit.

Postgres:

1. Run the Docker command from the reference section above
2. Create a `.env` file with the `DATABASE_URL`
3. Run `pnpm prisma db push` to verify the connection
4. Run `pnpm prisma studio` to verify the Prisma Studio UI opens

Prompt Checklist — your CLAUDE.md generation prompt should make Claude aware of:

- Paste the relevant output from your Step 1 planning conversation (conventions, folder structure, rules)
- The TaskCo stack: TypeScript, Fastify, Prisma, PostgreSQL, Zod, bcrypt, jose, Vitest
- The response envelope: `{ data: ... }` on success, `{ error: ... }` on failure
- The ownership rule: every database query must scope by ownerId
- The “never do” rules: never store plaintext passwords, never return passwordHash in responses, never build out-of-scope features
- The full out-of-scope list
- That you want 30–80 lines, not a comprehensive project document

Self-Review Checklist — before accepting the generated CLAUDE.md, check:

- Does it accurately list the stack that's actually installed (check package.json)?
- Does it include the folder structure that actually exists (check the file tree)?
- Does it include the ownership rule?
- Does it include the out-of-scope list?
- Is it 30–80 lines? If Claude wrote 150 lines, iterate: “Trim this to under 80 lines. Keep only conventions and rules. Remove anything Claude can infer from reading the code”
- Does it match your actual decisions from planning, or did Claude slip in opinions you didn't agree to?

Git:

1. Create a `.gitignore` (node_modules, .env, dist)
2. `git init`
3. `git add .` — verify with `git status` that you're not committing `.env` or `node_modules`

4. First commit

Final Checkpoint:

- `pnpm dev` starts Fastify and responds to HTTP requests
- `pnpm prisma studio` opens and connects to PostgreSQL
- `CLAUDE.md` exists at the project root, 30–80 lines, with stack, conventions, ownership rule, and out-of-scope list
- `.env` is in `.gitignore` and NOT committed
- First Git commit is clean

MODULE 5 — Auth Reference Card

Auth Quick Reference

Keep this section open during Lab 2. These are the spec details you'll reference in every prompt.

User model (Prisma)

```
User
  id          String    @id @default(cuid())
  email       String    @unique
  passwordHash String
  name        String
  createdAt   DateTime  @default(now())
  projects    Project[] ← added in Lab 3, do NOT include yet
```

The field is `passwordHash`, not `password`. Deliberate — the name prevents accidental plaintext storage.

Auth endpoints

Method	Path	Input	Success	Error
POST	<code>/auth/register</code>	email, password, name (all required)	<code>{ data: { token, user } }</code>	<code>{ error: "..." }</code>
POST	<code>/auth/login</code>	email, password (both required)	<code>{ data: { token, user } }</code>	<code>{ error: "..." }</code>

The `user` object in the response includes `id`, `email`, `name`, and `createdAt`. It never includes `passwordHash`.

Library responsibilities

Concern	Library
Input validation	Zod
Password hashing	bcrypt
JWT creation	jose
Database access	Prisma
HTTP framework	Fastify
Testing	Vitest

Traps active in Lab 2

- **Trap #1 — No existing code.** Fires in Step 3: reference the register file when building login.
- **Trap #3 — Literal words.** Fires in Steps 1-2: be specific about field types, constraints, and defaults.
- **Trap #5 — Didn't read the answer.** Fires in Step 2: read Claude's assumptions about JWT expiry and response shape.

Lab 2 — Auth (Part 1)

Day 1 ends after this lab. You'll finish auth in Module 6 tomorrow morning.

By the end of Lab 2 Part 1: User model in Prisma, register endpoint, login endpoint, basic tests passing.

By the end of Lab 2 Part 1, you should NOT have: JWT middleware, the `/me` endpoint, or comprehensive test coverage. Those are tomorrow.

Step 1 — User Model (~10 min)

Goal: Add the User model to your currently empty Prisma schema. Generate and apply a migration.

Prompt Checklist — your prompt should make Claude aware of:

- That you're adding to the existing `schema.prisma` file (not creating a new one)
- The fields, types, and constraints from the User model spec above — check each field's type, default value, and uniqueness
- That the password field is named to reflect that it stores a hash, not plaintext — deliberate naming, not a typo
- That the Project model does NOT exist yet — do NOT add a `projects` relation field to User right now. If Claude adds it, the migration will fail
- That you want a migration generated and applied (`prisma migrate dev`)
- The migration name (e.g., "add-user-model")

Self-Review Checklist — before accepting:

- Is the field called `passwordHash` ? If Claude named it `password` or `hashedPassword` , your prompt wasn't specific enough (*Trap #3*)
- Is `email` marked `@unique` ?
- Are defaults correct: `@default(cuid())` on id, `@default(now())` on createdAt?
- Did Claude add extra fields you didn't ask for (updatedAt, role, avatar, isActive)? (*Trap #3*)
- Did the migration apply cleanly?

Checkpoint: `pnpm prisma studio` shows a User table with the correct columns. The table is empty.

Step 2 — Register Endpoint (~20 min)

Goal: Build `POST /auth/register` — validates input with Zod, hashes the password with bcrypt, creates the user in Prisma, returns a JWT with a user object (minus passwordHash).

Before you prompt: Confirm your `.env` file has a `JWT_SECRET` entry (any long random string). If it's missing, Claude's register endpoint will crash at runtime.

Refer to the Good Prompt Structure template from Module 3. This is the first prompt where every section of that template earns its keep.

Prompt Checklist — your prompt should make Claude aware of:

- Which file the route goes in and how it's registered with Fastify
- That you're using Fastify, not Express — the route handler signature is different
- The Zod validation schema: email (must be valid email format), password (minimum length — pick a number, 8 is standard), name (required, non-empty)
- That the password must be hashed before storage — which library, when in the flow
- That a JWT is issued on successful registration — which library (check your package.json), what fields in the payload, where the signing secret comes from (not hardcoded)
- The JWT expiry: pick a duration and state it explicitly — don't let Claude choose for you
- The success response shape: the `{ data }` envelope from your CLAUDE.md, user object excluding sensitive fields
- The error responses: shape on validation failure, shape on duplicate email, which HTTP status codes
- That this is a standalone prompt — tests come later in Step 4

Self-Review Checklist — before accepting:

- Is the password hashed *before* the database call? If the raw password hits the database, something went wrong
- Does the response exclude sensitive fields? Check how Claude omitted them — make sure they're actually excluded
- Does the response match the `{ data }` envelope from your CLAUDE.md?
- Did Claude mention any assumptions? Look for paragraphs about JWT expiry, password requirements, error messages. **Read them.** (Trap #5)
- Does the code import libraries that are in your package.json?
- Did Claude add features you didn't ask for? (Trap #10)

Checkpoint: `POST` to `/auth/register` with `{ "email": "test@test.com", "password": "password123", "name": "Test User" }` returns `{ data: { token: "...", user: {...} } }`. The user appears in Prisma Studio. The passwordHash column is not the plaintext password.

Step 3 — Login Endpoint (~15 min)

Goal: Build `POST /auth/login` — validates credentials, verifies the password with bcrypt, returns a JWT.

This is the first time Trap #1 matters. You now have an existing auth route file. If your prompt doesn't reference it, Claude writes login from scratch in a potentially different style.

Prompt Checklist — your prompt should make Claude aware of:

- That the register endpoint already exists in a specific file — Claude should follow its patterns (Trap #1 — *reference the file by path*)
- The input: email and password only (no name)

- How password verification should work (the same library that hashed them should verify them)
- The success response: same shape as register
- The error response for invalid credentials: generic message or specific message? Decide and state it
- That JWT generation should follow the same approach as register (same secret, same expiry, same payload)

Self-Review Checklist — before accepting:

- Does login's code structure match register's? Same import style, same error handling, same response shape? If they look like different developers wrote them, Claude didn't reference the register file (*Trap #1*)
- Is password verification using the hashing library's compare function (not raw string comparison)?
- Did Claude add features you didn't ask for — account lockout, login attempt tracking? (*Trap #3*)
- Does the JWT use the same secret and expiry as register?

Checkpoint: POST to `/auth/login` with valid credentials returns the same response shape as register. Invalid credentials return `{ error: "..."}` .

Step 4 — Auth Tests (~15 min)

Goal: Tests for both register and login endpoints — success cases and key failure cases.

Prompt Checklist — your prompt should make Claude aware of:

- That you're using Vitest (not Jest — the syntax is similar but the imports and config differ)
- Where test files live in your project
- That you want Fastify's `inject` method for HTTP testing (not supertest, not a live server)
- The specific success and failure scenarios you expect for each endpoint — at least as many failure cases as success cases
- That tests need database cleanup between runs so they don't interfere with each other
- That the test file should build a Fastify app instance with the auth routes registered

Self-Review Checklist — before accepting:

- Does the test file import from `vitest` (describe, it, expect), not `jest`?
- Does it use `app.inject()` for HTTP calls?
- Are there both success and failure tests for register AND login?
- Does the test setup handle database cleanup?
- **Do the tests actually pass?** Run `pnpm test`. If they fail, read the error message carefully — iterate on the specific failure, don't throw away the test file (*Trap #9*)
- Did Claude write assertions that test the *response shape*, not just the status code?

Checkpoint: `pnpm test` passes. Green output for register success, register failures, login success, and login failures.

End-of-Day Checkpoint

- ☐ User model exists in Prisma with correct fields (passwordHash, email unique, defaults)
- ☐ `POST /auth/register` works — creates user, hashes password, returns JWT in `{ data }` envelope
- ☐ `POST /auth/login` works — verifies credentials, returns JWT in `{ data }` envelope
- ☐ `pnpm test` passes with tests for both endpoints (success and failure cases)
- ☐ Your code is committed to Git
- ☐ Login and register route files follow the same patterns (imports, error handling, response shapes)

If any of these fail, flag the instructor before leaving. Tomorrow morning starts with finishing auth, and a broken register or login blocks everything that follows.

MODULE 6 — Lab Reference

Quick reference: auth completion spec

Piece	Details
JWT middleware	Fastify preHandler hook. Reads <code>Authorization: Bearer <token></code> header. Verifies with <code>jose</code> . Attaches user info to request. Returns 401 on missing/invalid/expired token.
GET /auth/me	Protected endpoint (requires JWT middleware). Returns current user as <code>{ data: { id, email, name, createdAt } }</code> . Excludes <code>passwordHash</code> .
Token structure	Must match what your login endpoint issues. Same library, same secret, same claims.

Quick reference: Project model

```
Project
  id      String   @id @default(cuid())
  name    String
  description String?
  color   String   @default("#3b82f6")
  ownerId String
  owner   User     @relation(fields: [ownerId], references: [id])
  tasks   Task[]   ← added in Lab 4, do NOT include yet
  createdAt DateTime @default(now())
```

The User model needs a `projects Project[]` relation added as well.

Lab 2 finish — JWT middleware

Goal: Build JWT verification middleware that protects routes, then build the `/me` endpoint that uses it.

Prompt Checklist

Your prompt for the JWT middleware should make Claude aware of:

- Your existing login endpoint as reference — so the middleware matches the token it issues (library, algorithm, claims shape, secret location) (*Trap #1*)
- That you're building a Fastify preHandler hook, not Express-style middleware
- What user information to attach to the request object, and the shape it should take
- How to handle the three failure cases: missing Authorization header, malformed/invalid token, expired token
- The error response shape for each failure case (status code and `{ error }` envelope)

- Whether the middleware should query the database for the full user record or use the claims embedded in the token

Traps active

- **Trap #5** — Read Claude's assumptions about expiry, attachment shape, and error handling before accepting
- **Trap #1** — Show Claude your login endpoint so the middleware matches its token format

Self-Review Checklist

- Did Claude state any assumptions? Are they correct? (*Trap #5*)
- Does the middleware use the same JWT library and verification approach as your login endpoint? (*Trap #1*)
- Does it handle all three failure cases (missing, invalid, expired) with appropriate status codes?
- What did Claude attach to the request object? Is it what you wanted?
- Does the error response match your `{ error }` envelope pattern?

Checkpoint

Register a user, log in, use the returned JWT to hit a protected route. Try the same route with no token, a garbage token, and an expired token. All four cases should behave correctly.

Lab 2 finish — GET /auth/me

Goal: Build the endpoint that returns the currently authenticated user's profile.

Prompt Checklist

Your prompt for the /me endpoint should make Claude aware of:

- That this route uses the JWT middleware you just built
- Which user fields to include in the response and which sensitive fields must be excluded
- Your existing response envelope pattern
- Your existing route file structure — which file this goes in, how routes are registered
- That this is the simplest possible authenticated endpoint and can serve as the reference pattern for every protected route that follows

Self-Review Checklist

- Is `passwordHash` excluded from the response? (Check the Prisma `select` — don't just look at the type.)
- Does the route use the JWT middleware as a preHandler?
- Does the response shape match `{ data: { id, email, name, createdAt } }`?
- Did Claude follow your existing route registration pattern? (*Trap #1*)

Checkpoint

Log in, call GET /auth/me with the JWT. You see your user profile without the password hash. Calling without a token returns 401.

Lab 2 finish — Comprehensive auth tests

Goal: Tests that cover the full auth surface — not just happy paths.

Prompt Checklist

Your prompt for auth tests should make Claude aware of:

- Your test framework (Vitest) and any existing test patterns or helpers
- That you want tests for *all* of the following:
 - Register with valid input
 - Register with duplicate email (expect error)
 - Register with missing required fields (expect validation error)

- Login with valid credentials
- Login with wrong password
- Login with non-existent email
- GET /me with valid token
- GET /me with no token (expect 401)
- GET /me with expired token (expect 401)
- GET /me with malformed token (expect 401)
- Your test database strategy — how tests isolate from each other
- Whether to organize tests in one file or split by endpoint

Traps active

- **Trap #5** — Read Claude's assumptions about test setup and cleanup strategy
- **Trap #9** — If the first batch is 80% right, iterate ("add tests for expired tokens") — don't restart

Self-Review Checklist

- Do the tests cover error paths, not just happy paths? Count them — you should have at least as many error tests as success tests
- Do the tests actually assert the right things?
- Does the test for expired tokens actually use an expired token, not just no token? (Different failure modes.)
- Is there a cleanup strategy so tests don't pollute each other?

Checkpoint

Run `pnpm test`. All auth tests should pass. Read the test names — do they describe what they're testing, or are they generic?

Lab 3 start — Project model

Goal: Add the Project model to your Prisma schema and run the migration.

Prompt Checklist

Your prompt for the Project model should make Claude aware of:

- The exact fields and types (see the quick reference above)
- That the User model needs a `projects` relation field added
- Your existing User model as the pattern reference for field naming and style
- That a `Task` relation is declared now but the Task model doesn't exist yet — Claude should handle this gracefully
- That a Prisma migration is needed after schema changes

Self-Review Checklist

- Does the schema match the spec? Check field names, types, defaults, and optionality
- Is the relation between Project and User correctly defined on both sides?
- Did Claude add, remove, or rename anything you didn't ask for? (*Trap #10*)

Checkpoint

Run `pnpm prisma migrate dev`. The migration applies cleanly. Run `pnpm prisma studio` and confirm the Project table exists with the correct columns.

Lab 3 start — POST /projects

Goal: Create a new project owned by the authenticated user.

Prompt Checklist

Your prompt for POST /projects should make Claude aware of:

- That the endpoint requires authentication (uses your JWT middleware)

- That `ownerId` comes from the authenticated user on the request (`request.user`), NOT from the request body — this is a security constraint, not optional
- Your existing route patterns — show Claude a working route file (auth routes) (*Trap #1*)
- Zod validation for the request body — which fields are required, which are optional, what defaults apply
- The response envelope: `{ data: { project } }` on success, `{ error }` on validation failure

Traps active

- **Trap #1** — Show Claude your auth routes so project routes match the pattern
- **Trap #5** — Read Claude's assumptions about route location and ownerId handling

Self-Review Checklist

- Does the ownerId come from `request.user` , not from `request.body` ? This is the most important check.
- Is the route protected by your JWT middleware as a preHandler?
- Did Claude follow the same route structure as your auth routes? (*Trap #1*)

Checkpoint

Log in, create a project with POST `/projects` using a valid JWT. Check the database — the project's `ownerId` should match your user's ID.

Lab 3 start — GET `/projects`

Goal: List the current user's projects.

Prompt Checklist

Your prompt for GET `/projects` should make Claude aware of:

- That this returns ONLY projects where `ownerId` matches the authenticated user — not all projects in the database
- That this endpoint requires authentication
- Your existing route patterns (the POST `/projects` route you just built is now the closest reference)
- The response envelope: `{ data: { projects: [...] } }`
- Whether to include related data (task counts) or just the project fields

The ownership filter is the critical context here. If your prompt says "list projects" without mentioning per-user scoping, Claude will write `prisma.project.findMany()` with no `where` clause. That returns every project in the database.

Self-Review Checklist

- Does the query filter results to only the authenticated user's projects? (*Trap #2 preview*)
- Is the endpoint protected by the JWT middleware?
- Did Claude add anything you didn't ask for — pagination, sorting, extra endpoints? (*Trap #10*)

Checkpoint

Create two users. Create projects with each. Call GET `/projects` with User A's token. You should see only User A's projects.

Test-driven approach (optional, for those ahead)

If you finish the Lab 2 tests early and want to try the test-driven approach for Lab 3:

1. Write a failing test for POST `/projects` before writing the endpoint. Assert that creating with a valid JWT returns 200 with the right shape, and without a JWT returns 401.
2. Run the test — it should fail (no endpoint yet).
3. Prompt Claude to make the test pass, showing it the failing test as context.

4. Repeat for GET /projects with an ownership-scoped test: create projects for two users, assert that GET only returns the authenticated user's projects.

MODULE 7 — Lab Reference

Quick reference: SYMPTOM / CODE / TRIED / ASK

Use this template every time you're debugging with Claude. Pasting only the error is Trap #6.

Part	What to include	What to leave out
SYMPTOM	Exact error text or exact unexpected behavior. Copy-paste, don't paraphrase.	Your theory about <i>why</i> it's happening — that goes in ASK
CODE	The specific function or block involved. If the error points to a line, show that line plus 10–15 lines of context.	The entire file. Point Claude there if it needs more.
TRIED	What you already checked or attempted. Be specific: "I confirmed the JWT is valid by hitting /auth/me first"	Things you haven't actually verified. Don't bluff — Claude trusts your TRIED section.
ASK	A specific question with a specific answer.	"Why is this broken?" — that's an invitation to guess.

Quick reference: asking specific questions about code

Good questions have a yes/no or specific factual answer:

- "Does this handler return a response on every code path?"
- "What happens if `projectId` doesn't match any row?"
- "Is there a path where the error envelope doesn't match `{ error: string } ?`"
- "Does this Prisma query include the ownership filter?"

Use this (Pattern 2) when reviewing Claude's output before accepting, and when tracing bugs through multiple files.

Lab 3 (continued) — Finish projects CRUD

Goal

Build the remaining three project endpoints: PATCH /projects/:id, DELETE /projects/:id, and GET /projects/:id. Every endpoint must verify that the requesting user owns the project. By the end of this lab, all five project endpoints work and pass the ownership test.

What you have so far

From the Lab 3 start in Module 6, you should have:

- Project model in Prisma schema
- POST /projects — creates a project with the authenticated user as owner
- GET /projects — lists projects (this may or may not already filter by owner — that's about to matter)

Endpoint specs

PATCH /projects/:id

- Accepts partial body: any combination of `name`, `description`, `color`
- Validates with Zod (all fields optional for partial update)

- Verifies the authenticated user owns the project before updating
- Returns `{ data: updatedProject }` on success
- Returns 403 or 404 if the user doesn't own the project (pick one and be consistent across all endpoints)

DELETE /projects/:id

- Verifies the authenticated user owns the project before deleting
- Deletes the project and its tasks (Prisma cascade or explicit delete)
- Returns `{ data: { deleted: true } }` or similar confirmation
- Returns 403 or 404 if the user doesn't own the project

GET /projects/:id

- Verifies the authenticated user owns the project
- Returns the project with a task count (Prisma `_count` or manual count)
- Returns `{ data: project }` with the task count included
- Returns 403 or 404 if the user doesn't own the project or it doesn't exist

Prompt Checklist — PATCH /projects/:id

Your prompt should make Claude aware of:

- The file where your existing project routes live (show Claude the file or point it there)
- That this is a partial update — Zod validation should accept any subset of fields, not require all of them
- That the endpoint must verify ownership before performing the update
- Your existing response envelope pattern
- What the error response should look like when the user doesn't own the project
- Whether you want the updated project returned or just a success flag

Self-Review Checklist — PATCH /projects/:id

- Does the route look up the project and verify `ownerId === userId` before updating? (*Trap #2*)
- Does the Zod schema use `.partial()` or equivalent so single-field updates don't fail validation?
- Does the response shape match your existing endpoints? (*Trap #1*)
- If the response was mostly right but something's off, are you iterating or starting over? (*Trap #9*)

Prompt Checklist — DELETE /projects/:id

Your prompt should make Claude aware of:

- Your existing route file
- That the endpoint must verify ownership before deleting
- That deleting a project should also remove its tasks
- Your response envelope pattern for the success case and error case

Self-Review Checklist — DELETE /projects/:id

- Does the route verify ownership before calling delete? (*Trap #2*)
- Are tasks deleted when the project is deleted?
- Did Claude follow your existing route file's pattern? (*Trap #1*)
- If you hit an error, did you use SYMPTOM / CODE / TRIED / ASK or did you just paste the error? (*Trap #6*)

Prompt Checklist — GET /projects/:id

Your prompt should make Claude aware of:

- Your existing route file
- That the endpoint must verify the requesting user owns this specific project
- That the response should include a count of tasks belonging to the project
- Your response envelope pattern
- Your error response for non-existent or non-owned projects
- How you want the task count represented

Self-Review Checklist — GET /projects/:id

- Does the query scope to the authenticated user, not just find by ID? (Trap #2)
- Is the task count actually included in the response?
- Does the 404/403 case return your standard error envelope?
- Did Claude reference any library or Prisma feature you haven't used before? If so, verify it exists. (Trap #7)

The ownership test

Once all three endpoints are built, run this test. Write it yourself or ask Claude to generate it — but read the test carefully before running it. This test is designed to catch Trap #2.

What the test should do:

1. Create User A (register via POST /auth/register)
2. Create User B (register via POST /auth/register with different credentials)
3. User A creates a project (POST /projects with User A's JWT)
4. **User B lists projects** (GET /projects with User B's JWT) — should return **zero projects**
5. **User B tries to GET User A's project** (GET /projects/:id with User B's JWT) — should return 403 or 404
6. **User B tries to PATCH User A's project** — should return 403 or 404
7. **User B tries to DELETE User A's project** — should return 403 or 404

If step 4 returns User A's project, your GET /projects endpoint is not filtering by owner. That's Trap #2.

If steps 5–7 succeed instead of returning an error, the individual endpoints aren't checking ownership. Same trap, same fix.

Prompt Checklist — generating the ownership test

Your prompt for the ownership test should make Claude aware of:

- Your existing test file structure (show Claude a test that already exists — the auth tests from Lab 2)
- That the test needs two separate users with separate JWTs
- That you're using Vitest
- The specific assertions: User B should see zero projects and get errors on all single-project operations
- Your response envelope pattern so assertions check the right shape
- That this test is specifically designed to verify ownership isolation

Self-Review Checklist — the ownership test

- Does the test actually create two *different* users, or does it reuse the same credentials?
- Does step 4 assert `data.length === 0`, not just `status === 200`?
- Do steps 5–7 assert the correct error status code?
- Does the test clean up after itself, or will it fail on a second run because the email is already taken?

Traps active in this lab

- **Trap #2** — If your endpoint returns all projects, your prompt missed ownership scoping. The ownership test catches this.
- **Trap #6** — When debugging, use SYMPTOM / CODE / TRIED / ASK. Don't paste just the error.
- **Trap #9** — If the output is 80% right, iterate — don't restart.

Lab 3 Checkpoint

- ☐ POST /projects creates a project owned by the authenticated user
- ☐ GET /projects returns only the current user's projects (not all projects)
- ☐ GET /projects/:id returns the project with task count, only if owned by the user
- ☐ PATCH /projects/:id updates the project, only if owned by the user
- ☐ DELETE /projects/:id deletes the project and its tasks, only if owned by the user

- [?] The ownership test passes: User B cannot see, modify, or delete User A's projects
- [?] All responses use the `{ data }` / `{ error }` envelope consistently

If your ownership test is still failing, don't move on. This is the foundation for everything in Labs 4 and 5.